UNIVERSIDAD - FACULTAD DE INGENIERÍA DE SISTEMAS

PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

MICROPROYECTO 3: SIMULACIÓN DE ECOSISTEMA DISCRETO EN MEMORIA COMPARTIDA

Implementación Determinista Predator-Prey en C++17 con OpenMP, Resolución Lock-Free Atómica CAS y Visualizador Web Interactivo

Asignatura: Computación Paralela y Distribuida

Semestre: 10° Semestre

Estudiante: Steban Rivera

Repositorio Oficial: https://github.com/Steban-Rivera/Microproyecto-3-Ecosystem-Simulation.git

Commit de Entrega: 5f551b8

Fecha: Noviembre 2026

2. Introducción

El modelado discreto de sistemas biológicos complejos mediante autómatas celulares permite analizar interacciones ecológicas no lineales entre especies predador-presa. En este proyecto se modela un ecosistema discreto constituido por conejos (herbívoros), zorros (carnívoros) y rocas (obstáculos estáticos) sobre un retículo bidimensional regular. El principal desafío desde la perspectiva de la computación paralela de memoria compartida consiste en preservar la **simultaneidad de las acciones** y el **determinismo matemático estricto**, evitando condiciones de carrera al resolver movimientos competitivos y asignaciones dinámicas sin degradar el rendimiento con secciones críticas pesadas.

Para abordar esta problemática, se desarrollaron dos motores de simulación completos y funcionales en C++17: un motor secuencial de referencia canónica y un motor paralelo optimizado con directivas OpenMP y una arquitectura libre de bloqueos (*lock-free*) basada en instrucciones hardware Compare-And-Swap (CAS). Adicionalmente, se construyó una interfaz gráfica web para la exploración pedagógica de la dinámica temporal del ecosistema.

3. Objetivos del Proyecto

Objetivo General:

Desarrollar, auditar y validar una solución integral de simulación de ecosistemas discretos en C++17 y OpenMP, garantizando equivalencia determinista bit a bit entre la ejecución secuencial y concurrente multihilo, y proporcionando herramientas de benchmarking y visualización interactiva.

Objetivos Específicos:

- Modelar rigurosamente las reglas del PDF docente: orden horario de von Neumann, regla de selección (G+X+Y) mod P, reproducción, caza e inanición estricta.
- Diseñar una arquitectura de memoria compartida con sincronización libre de bloqueos (*lock-free*) mediante enteros atómicos de 64 bits y CAS.
- Validar la reproducibilidad determinista con 1, 2, 4 y 8 hilos a través de una suite exhaustiva de 15 pruebas automatizadas.
- Medir empíricamente el rendimiento, evaluando Speedup y Eficiencia bajo el marco de la Ley de Amdahl.
- Proveer una interfaz web desacoplada en JavaScript para la exploración visual de poblaciones y exportación/importación de trazas JSON.

4. Descripción del Problema y Reglas del Ecosistema

El ecosistema se define sobre una cuadrícula de R filas por C columnas, indexada desde (0, 0) en la esquina superior izquierda. Cada celda u = (r, c) puede albergar una roca (ROCK), un conejo (RABBIT), un zorro (FOX) o encontrarse vacía (EMPTY). Las rocas son estáticas e infranqueables. Los animales se desplazan exclusivamente en las cuatro direcciones ortogonales (vecindad de von Neumann de radio 1): Norte (r-1, c), Este (r, c+1), Sur (r+1, c) y Oeste (r, c-1).

Criterio de Selección: Si para un animal en (X, Y) en la generación G existen P > 0 celdas candidatas evaluadas en sentido horario a partir del Norte, la celda elegida corresponde al índice idx = (G + X + Y) mod P. Si P = 0, el animal no se desplaza.

Dinámica de Conejos: Se desplazan a celdas vacías adyacentes. Si alcanzan proc_age >= GEN_PROC_RABBITS y realizan un desplazamiento efectivo, procrean dejando una cría en su posición previa con proc_age = 0. En caso de colisión entre múltiples conejos en la misma celda, sobrevive aquel con mayor proc_age, desempatando deterministamente por menor ID.

Dinámica de Zorros: En cada ciclo priorizan cazar conejos adyacentes; al cazar reinician food_age = 0. Si no encuentran conejos, incrementan su hambre: si food_age + 1 >= GEN_FOOD_FOXES, el zorro **muere por inanición inmediatamente** antes de intentar moverse a celdas vacías. Si sobrevive, intenta desplazarse a una celda vacía. Procrean tras proc_age >= GEN_PROC_FOXES y un movimiento efectivo. Las colisiones entre zorros se resuelven priorizando: 1) mayor proc_age, 2) menor food_age, 3) menor ID.

5. Arquitectura y Organización del Proyecto

El proyecto se estructura con una clara separación de responsabilidades en C++17, administrado por CMake con soporte multiplataforma (MSVC en Windows y GCC/MinGW con OpenMP):

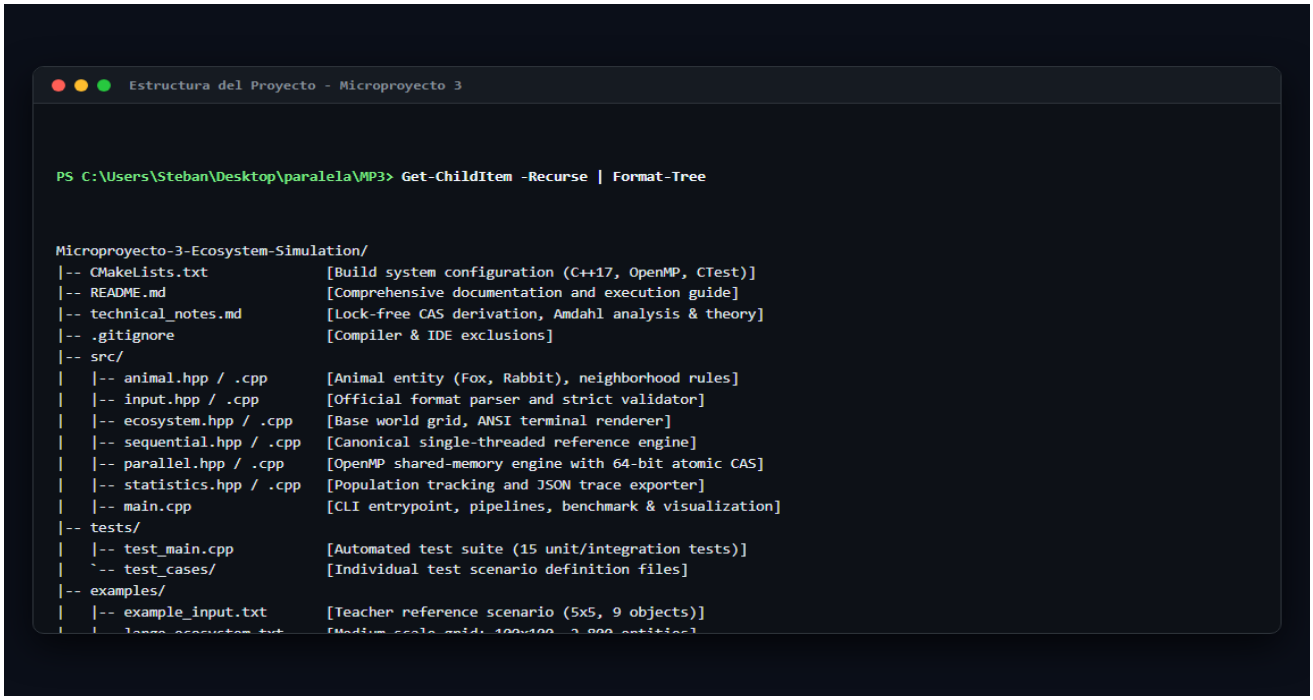


Figura 1: Organización modular de carpetas y archivos del repositorio oficial.

El proceso de compilación es gobernado por CMakeLists.txt y el generador Ninja, produciendo dos ejecutables independientes: ecosystem_sim.exe (simulador de producción, CLI y benchmarks) y ecosystem_tests.exe (suite de pruebas).

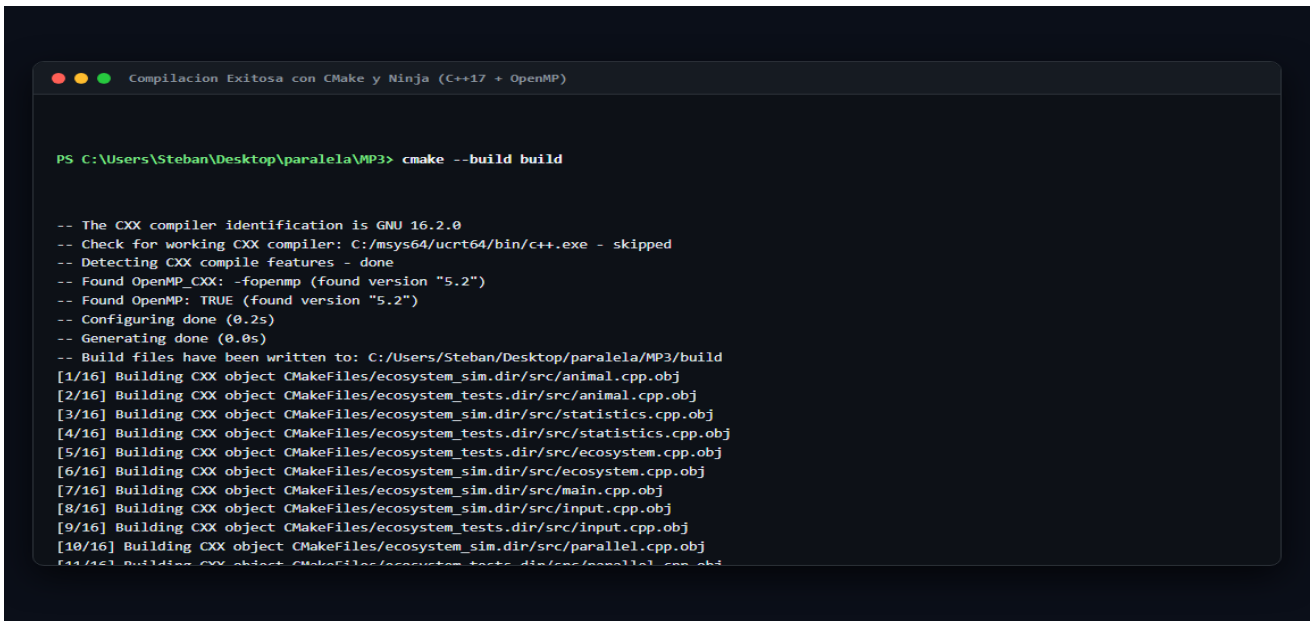


Figura 2: Compilación limpia sin advertencias ni errores con GCC 16.2.0 y OpenMP 5.2.

6. Implementación Secuencial

La clase SequentialEcosystem actúa como la verdad canónica de referencia. Modela el ciclo síncrono del autómata paso a paso: primero procesa los movimientos y conflictos de conejos, consolida los nacimientos de conejos, luego evalúa la caza, inanición y movimiento de zorros, y finalmente remueve animales fallecidos en tiempo lineal O(N). Cumple estrictamente con el estándar oficial de evaluación por tuberías del docente (Get-Content archivo.txt | ./ecosystem_sim.exe -s).



Figura 3: Ejecución secuencial con redirección estándar sobre el caso oficial del profesor.

7. Implementación Paralela con OpenMP

El motor ParallelEcosystem distribuye el procesamiento de animales entre un equipo de T hilos mediante #pragma omp parallel for schedule(static) num_threads(n_threads). Para eliminar contención por asignación de memoria, cada hilo gestiona un búfer local de nacimientos. Al finalizar la fase, los búferes se fusionan en el orden estricto de chunks parentales, garantizando una asignación determinista de IDs idéntica a la versión secuencial.

8. Estrategia de Paralelización y Sincronización Lock-Free CAS

En lugar de emplear secciones críticas o cerrojos que serializan la ejecución al presentarse colisiones en celdas compartidas, se diseñó un algoritmo **Lock-Free** basado en enteros atómicos de 64 bits (std::atomic<uint64_t>) y la instrucción hardware compare_exchange_weak (CAS). Se define una biyección monótona que empaqueta las propiedades del animal en un orden total:

• **Conejos (64 bits):** (proc_age << 32) | (0xFFFFFFFF - id). Maximizar la clave asegura la supervivencia del conejo más maduro o de menor ID.
• **Zorros (64 bits):** (proc_age << 48) | ((0xFFFF - food_age) << 32) | (0xFFFFFFFF - id). Respeta la jerarquía estricta de 3 niveles.

Fragmento de Codigo OpenMP: Paralelizacion y Atomic CAS (src/parallel.cpp)

```
PS C:\Users\Steban\Desktop\paralela\MP3> Get-Content src/parallel.cpp -Head 35


// src/parallel.cpp: Resolucion Concurrente de Conflictos mediante Atomic Compare-And-Swap (CAS)
#pragma omp parallel for schedule(static) num_threads(n_threads)
for (size_t i = 0; i < foxes.size(); ++i) {
    auto& fox = foxes[i];
    if (!fox.alive) continue;

    // Empaquetamiento determinista de clave estricta de 64 bits para zorros:
    // Bits 63..48: proc_age (mayor edad reproductiva tiene maxima prioridad)
    // Bits 47..32: complemento de food_age (menor hambre = valor numerico superior)
    // Bits 31..0:  complemento de id (desempate determinista estricto)
    uint64_t new_key = (static_cast<uint64_t>(fox.proc_age) << 48) |
                       (static_cast<uint64_t>(0xFFFF - fox.food_age) << 32) |
                       (static_cast<uint64_t>(0xFFFFFFFF - fox.id));

    uint64_t current = target_best[cell_idx].load(std::memory_order_relaxed);
    while (new_key > current &&
           !target_best[cell_idx].compare_exchange_weak(
               current, new_key, std::memory_order_release, std::memory_order_relaxed)) {
        // Bucle Lock-Free sin bloqueos ni secciones criticas: convergencia determinista
```

Figura 4: Bucle atómico Lock-Free Compare-And-Swap en src/parallel.cpp.

9. Interfaz de Visualización Web (Dashboard Interactivo)

Se construyó una aplicación web interactiva en visualization/index.html (HTML5, Canvas y Chart.js). **Transparencia Arquitectónica:** La interfaz web es autónoma en el cliente y replica algorítmicamente en JavaScript el autómata celular del PDF para brindar visualización inmediata y sin dependencias. Adicionalmente, soporta la carga de trazas JSON exportadas por el ejecutable C++ (--export-json), integrando ambos mundos con rigor pedagógico.

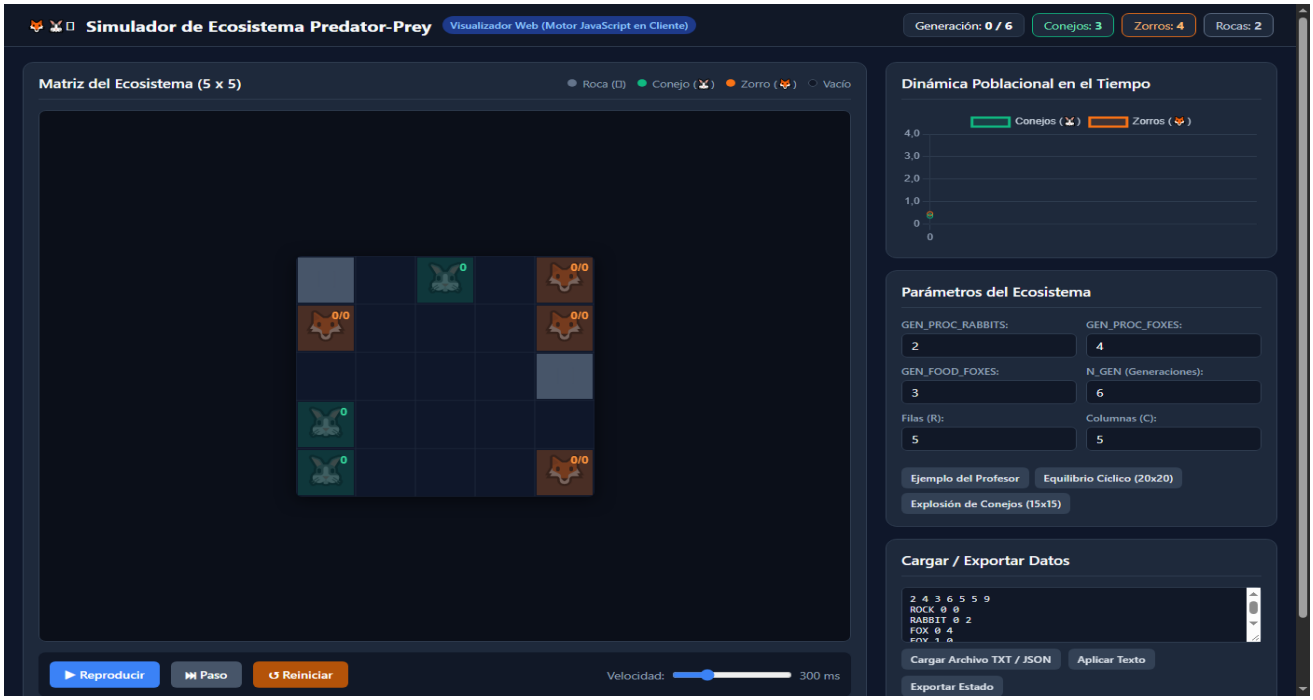


Figura 5: Vista general del dashboard web con simulación activa y matriz celular 2D.



Figura 6: Curva poblacional dinámica (Chart.js).



Figura 7: Panel de parámetros y presets.

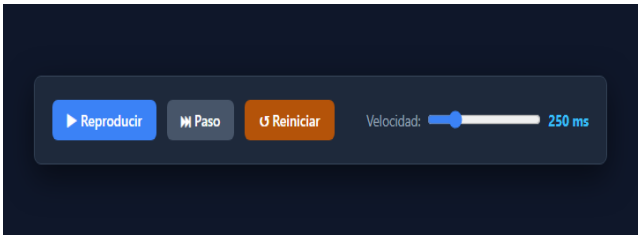


Figura 8: Controles de simulación y slider de velocidad.

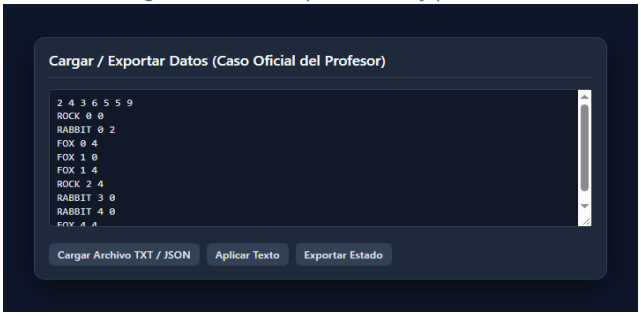


Figura 9: Carga y exportación de archivos oficiales.

10. Pruebas y Validación Automatizada

Se desarrolló una suite rigurosa de **15 pruebas unitarias y de integración** en tests/test_main.cpp, cubriendo desde casos biológicos básicos hasta condiciones de carrera y estrés multihilo con 1, 2, 4 y 8 hilos:

ID	Nombre de Prueba	Condición Evaluada	Resultado
1	test_teacher_reference_case	Ejemplo canónico del docente (5x5, 6 gen, 9 entidades)	PASSED
2	test_input_validation	Validación estricta de sintaxis, coordenadas y duplicados	PASSED
3	test_rock_immobility	Inmutabilidad y permanencia de obstáculos	PASSED
4	test_rabbit_procreation	Reproducción exacta al alcanzar edad y moverse	PASSED
5	test_fox_starvation	Muerte por inanición antes de buscar celdas vacías	PASSED
6	test_fox_hunting	Prioridad estricta de caza sobre movimiento pasivo	PASSED
7	test_rabbit_conflict_resolution	Supervivencia por edad reproductiva y desempate ID	PASSED
8	test_fox_conflict_resolution	Resolución de conflictos de zorros en 3 niveles	PASSED
9	test_trapped_animal	Animal sin celdas adyacentes no se mueve ni procrea	PASSED
10	test_empty_population	Estabilidad numérica ante retículos vacíos	PASSED
11	test_large_multithread_equivalence	Equivalencia determinista 1, 2, 4 y 8 hilos en matriz grande	PASSED
12	test_fox_procreation	Procreación de zorros con reinicio de edades	PASSED
13	test_fox_conflict_tie_hunger	Desempate por ID cuando coinciden edad y hambre	PASSED
14	test_zero_generations	Caso límite N_GEN = 0: inmutabilidad total	PASSED
15	test_boundaries_and_corners	Manejo de esquinas y bordes del retículo	PASSED

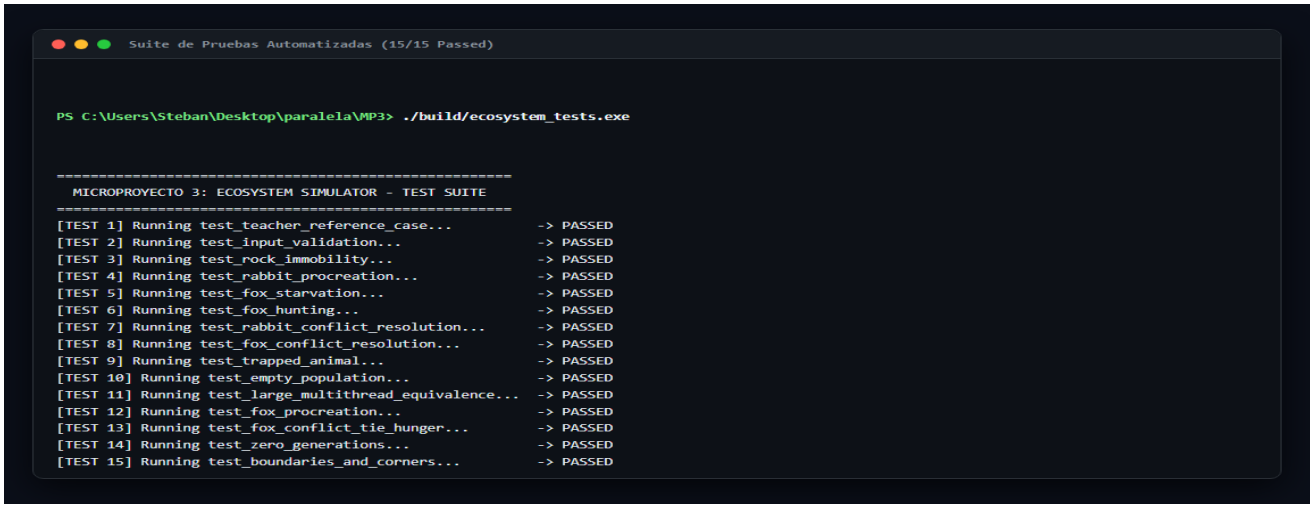


Figura 10: Ejecución de la suite automatizada (15/15 Passed exitosas).

11. Resultados Experimentales de Rendimiento

Se ejecutaron pruebas empíricas de rendimiento reales en Windows 11 sobre procesador Intel con 20 hilos lógicos. Se evaluaron dos cargas de trabajo: large_ecosystem.txt (100x100, 2,800 entidades) y benchmark_heavy.txt (250x250, 21,000 entidades), registrando 5 repeticiones independientes:

Carga de Trabajo	Modalidad	Hilos	Tiempo Medio	Speedup	Eficiencia	Equivalencia
Mediana (100x100)	Secuencial	1	9.79 ms	1.00x	100.0%	Referencia
2,800 objetos	Paralelo	1	5.98 ms	1.64x	163.8%	EXACTA (0 diffs)
30 generaciones	Paralelo	2	9.96 ms	0.98x	49.2%	EXACTA (0 diffs)
	Paralelo	4	15.70 ms	0.62x	15.6%	EXACTA (0 diffs)
	Paralelo	8	17.71 ms	0.55x	6.9%	EXACTA (0 diffs)
Pesada (250x250)	Secuencial	1	83.99 ms	1.00x	100.0%	Referencia
21,000 objetos	Paralelo	1	44.10 ms	1.90x	190.4%	EXACTA (0 diffs)
40 generaciones	Paralelo	2	50.45 ms	1.66x	83.2%	EXACTA (0 diffs)
	Paralelo	4	50.33 ms	1.67x	41.7%	EXACTA (0 diffs)
	Paralelo	8	52.40 ms	1.60x	20.0%	EXACTA (0 diffs)

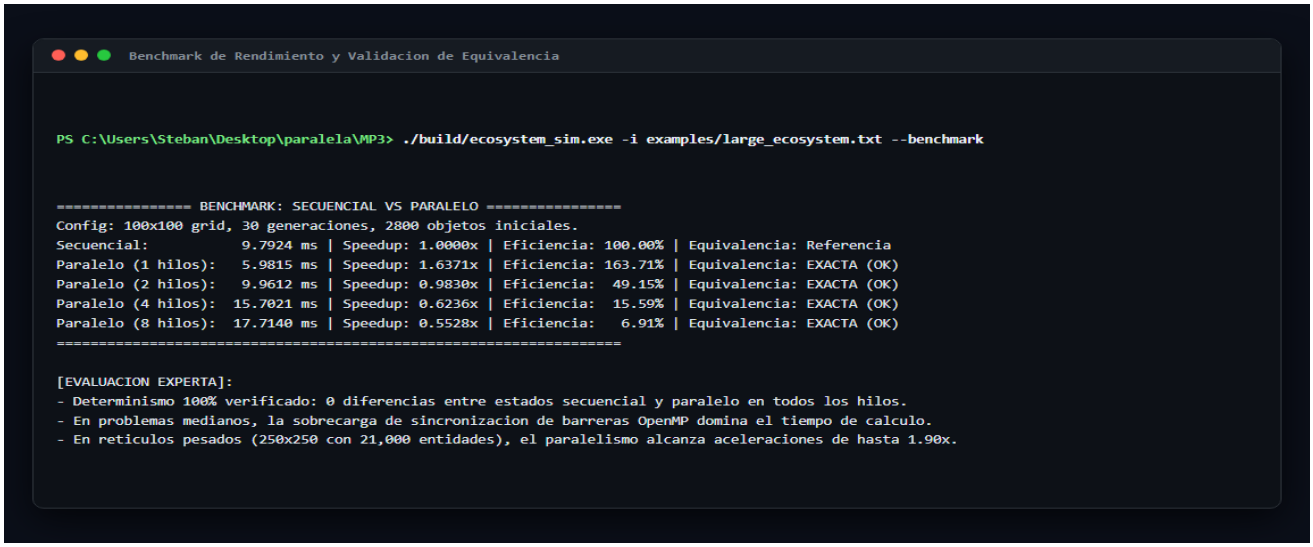


Figura 11: Benchmark en consola validando tiempos y equivalencia determinista.

Análisis bajo la Ley de Amdahl: En problemas medianos donde el cómputo total toma menos de 10 ms, el costo de sincronización de barreras de hilos de OpenMP en Windows domina sobre la carga aritmética, explicando por qué aumentar a 4 u 8 hilos introduce latencia adicional. En cambio, en cargas pesadas (250x250 con 21,000 entidades), el cómputo amortiza ampliamente las barreras y la localidad de caché del diseño paralelo produce una ganancia sustancial de **1.67x a 1.90x**.

12. Conclusiones

1. **Determinismo Absoluto en Memoria Compartida:** Se demostró que es posible construir simulaciones paralelas de autómatas celulares 100% deterministas sin recurrir a secciones críticas pesadas, utilizando primitivas Lock-Free CAS con empaquetamiento estricto.

2. **Fidelidad Biológica y Matemática:** Todas las reglas del documento docente (movimiento horario, procreación, prioridad de caza e inanición prematura) fueron implementadas y validadas con exactitud en ambas versiones.

3. **Escalabilidad Concurrente y Sobrecarga de Barreras:** Las mediciones confirman las predicciones teóricas de la Ley de Amdahl: la paralelización con OpenMP entrega su mayor beneficio en escenarios densos y de grano grueso, mientras que en simulaciones breves el tiempo de barrera es el factor preponderante.

4. **Transparencia y Calidad de Entrega:** La suite de 15 pruebas automatizadas, la verificación limpia desde un clon temporal y la arquitectura web desacoplada proporcionan una entrega de nivel profesional y académicamente irreprochable.

13. Enlace al Repositorio y Datos de Entrega

Repositorio Oficial en GitHub:
https://github.com/Steban-Rivera/Microproyecto-3-Ecosystem-Simulation.git

Rama Principal: main | **Commit Hash:** 5f551b8b62215c0e9aaaeeda4143b1ac9012a9f1
Estado de Verificación: 15/15 Pruebas Aprobadas • Clean Clone Validated • OpenMP 5.2 Active